CSD208- Computer Architecture
Assignment - MIPS PROGRAMMING
Total Marks – 5
Due date – 8th March 2019

1. Convert following MIPS code into its equivalent C code.

```
lw    $t0, 4($gp)
add   $t1, $t0, $zero
addi  $t1, $t1, 3
mult  $t1, $t1, $t0
sw    $t1, 0($gp)
```

```
int y;          //Assuming a variable y to be stored at location 4 in memory
int z=y+0;
z+=3;
z*=y;
int x=z;        // Assuming a variable x to be stored at location 0 in memory
```

2. Convert following MIPS code into its equivalent C code.

```
    lw    $t0, 0($gp)
    lw    $t1, 4($gp)
    slt   $t1, $t0, $t1
    beq   $t1, $zero, skip
    sll   $t0, $t0, 2
    add   $t0, $t0, $gp
    sw    $zero, 28($t0)
skip:
```

```
int x;          // Assuming a variable x to be stored at location 0 in memory
int y;          //Assuming a variable y to be stored at location 4 in memory
if(x<y)
        y=1;
else
        y=0;
if(y==0)
        goto skip;
else
{
        x*= Math.pow(2,2);          //Multiply value of x by 4
        x+= &x;                     //Adding address of x to x
        arr[x+(28/sizeof(int))]=0;  //arr is a random array
}
```

3. Read following MIPS program carefully and explain what is going in the program.

```
lw    $t0, 0($gp)
bltz  $t0, def
slti  $t1, $t0, 3
beq   $t1, $zero, def
```

```
        sll   $t0, $t0, 2
        add   $t2, $t0, $gp
        lw    $t2, 1064($t2)
        jr    $t2
is0: sw    $zero, 28($gp)
     j     done
is1:
is2:  addi  $t0, $zero, 1
      sw    $t0, 32($gp)
      j     done
def: addi  $t0, $zero, -1
        sw    $t0, 28($gp)
        j     done
done:
```

```
//The best explanation is to convert to an understandable C code

int x;
if (x < 0 || !(x < 3)) {
   switch (x) {
   case 0:
      a[0] = 0;
      break;
   case 1:
   case 2:
      a[1] = 1;
   }
} else {
   a[0] = -1;
}
```

4. Write MIPS instruction for following.
   i. For jumping to location 50 of memory

      j 50

   ii. A branch on register $s0 being equal to register $s1

      beq $s0,$s1,dest

   iii. To print your name.

```
   .data
name:      .asciiz     "Kabir Kanha Arora"
   .text
   .globl main
main:
   li     $v0, 4
   la     $a0, name
   syscall
   li     $v0, 10
   syscall
```

5. Write a program in MIPS to Compute the sum of N numbers. Take the value of N from user.

```
        .globl   main
main:
        li       $v0,4
        la       $a0, msg1
        syscall
        li       $v0,5
        syscall
        move     $t0,$v0
        li       $t1, 0
        li       $t2, 0
loop:   addi     $t1, $t1, 1
        add      $t2, $t2, $t1
        beq      $t0, $t1, exit
        j        loop
exit:   li       $v0, 4
        la       $a0, msg2
        syscall
        li       $v0,1
        move     $a0, $t2
        syscall
        li       $v0,4
        la       $a0, lf
        syscall
        li       $v0,10
        syscall
        .data
msg1:   .asciiz  "Number of integers (N)?  "
msg2:   .asciiz  "Sum = "
lf:     .asciiz  "\n"
```

6. Convert following C code into its equivalent MIPS code.

```
#include <stdio.h>
int function(int a);
int main()
{
  int x=5;
  int y;
  y = function(x);
    printf("y=%i\n", y);
  return 0;
}
int function(int a)
{
  return 3*a+5;
}
```

```
        .text

        .globl main
main:
        lw $t3, 0($gp)
        li $t2, 5
        jal func
```

```
        #Print y=
        li $v0, 4
        la $a0, ans
        syscall

        #Print value
        li $v0,1
        move $a0, $t3
        syscall

        #Print \n
        li $v0,4
        la $a0,newline
        syscall

        li $v0, 10
        syscall
        .data
func:
        mul $t2, $t2, 3
        addi $t3, $t2, 5
        jr $ra

        .data
ans:    .asciiz "y="
newline:        .asciiz "\n"
```

7. Explain MIPS Addressing Modes with example.

Register

- Register Addressing is considered the simplest addressing mode.
- This is because both operands are in a register. Which allow instructions to be executed much faster in comparison with other addressing modes because they does not involves with memory access.
- The number of registers is limited since only a few bits are reserved to select a register.
- Register Addressing is a form of direct addressing , this is because we are only interested in the number in the register , rather than using that number as a memory address.

Here's an example of Register Addressing :

```
add $s1 , $s2 , $s3 also means that $s1 ←$s2 + $s3

where ; $s1 = rd
            $s2 = rs
            $s3 = rt
```

Immediate

- Immediate Addressing is a numeric value embedded in the instruction in the actual operand.
- In immediate addressing , the operand is a constant within the encoded instruction.
- Immediate addressing has the advantage of not requiring an extra memory access to fetch the operand , hence will be executed faster. However , the size of operand is limited to 16 bits.

- The jump instruction format also falls under immediate addressing , where the destination is held in the instruction.
- Now an example of Immediate Addressing :

```
addi $t1 , $zero , 1 means $t1 ← 0 + 7

(add immediate , uses the I-type format)

where ; $t1 = rd
            $zero = r1
               1 = immediate value
```

PC-relative

- PC-Relative Addressing also known as Program Counter Addressing is a data or instruction memory location is specified as an offset relative to the incremented PC.
-
- PC-relative addressing is usually used in conditional branches. PC refers to special purpose register , Program Counter that stores the address of next instruction to be fetched.
- In PC-relative addressing , the offset value can be an immediate value or an interpreted label value.
- The effective address is the sum of the Program Counter and offset value in the instruction. The effective address determines the branch target.
- PC-relative addressing implements position-independent codes. Only a small offset is adequate for shorter loops.
- Branch instructions can only move 32768 above or below the program counter because the offset is a 16-bit two's complement number.
- Another word of saying to explain PC-Relative Addressing :
- The operand address = PC + an offset
- Implements position-independent codes. A small
- offset is adequate for short loops.

```
Example: beqz $t0 , strEnd
where ; $t0 = rs
            100 = offset
Thus ; if ($t1 == 0) goto PC + 4 + (4*2)
```

- In this instruction , beqz is a conditional instruction that branches to label in the code if the content of $t0 is equal to zero. If the current address for branch instruction in execution is 0x4000000C , the effective address will be 40000018.

Pseudo-Direct Addressing

- Pseudo-direct Addressing is the memory address which (mostly) embedded in the instructions.
-
- Pseudo-Direct addressing is specifically used for J-type instructions , j and jal. The instruction format is 6 bits of opcode and 26 bits for the immediate value (target).

- In Pseudo-Direct addressing , the effective address is calculated by taking the upper 4 bits of the Program Counter(PC) , concatenated to the 26 bit immediate value , and the lower two bits are 00.
- Therefore , the new effective address will always be a word-aligned and we can never have a target address of a jump instruction with the two bits anything other than 0 0 and creates a complete 32-bit address. Since the upper 4 bits of the PC are used, this constrains the jump target to anywhere within the current 256 MB block of code (1/16 of the total 4 GB address space). To jump anywhere within the 4 GB space, the R-type instructions jr and jalr are used , where the complete 32 - bit target address is specified in a register.
- *Note :
- Address in Pseudo-Direct must be a multiple of four.

Indirect addressing:

```
lw      $t2, ($t0)
```

- load word at RAM address contained in $t0 into $t2

```
sw      $t2, ($t0)
```

- store word in register $t2 into RAM at address contained in $t0

Based or indexed addressing:

```
lw      $t2, 4($t0)
```

- load word at RAM address ($t0+4) into register $t2
- "4" gives offset from address in register $t0

```
sw      $t2, -12($t0)
```

- store word in register $t2 into RAM at address ($t0 - 12)
- negative offsets are fine

Note: based addressing is especially useful for:

- arrays; access elements as offset from base address
- stacks; easy to access elements at offset from stack pointer or frame pointer

```
example

                .data
array1:         .space  12              #  declare 12 bytes of storage to hold array of 3
integers
                .text
__start:        la      $t0, array1             #  load base address of array into
register $t0
                li      $t1, 5          #  $t1 = 5   ("load immediate")
                sw $t1, ($t0)           #  first array element set to 5; indirect
addressing
```

```
li $t1, 13              #   $t1 = 13
sw $t1, 4($t0)          #  second array element set to 13
li $t1, -7              #   $t1 = -7
sw $t1, 8($t0)          #  third array element set to -7
done
```

8. What is the function of following MIPS Instruction?

i. SYSCALL

System Call

Generates a software interrupt.

ii. SRLV

Shift Right Logical Variable

Shifts a register value right by the amount specified in $s and places the value in the destination register. Zeroes are shifted in.

iii. SRL

Shift Right Logical

Shifts a register value right by the amount specified in $s and places the value in the destination register. Zeroes are shifted in.

iv. MULTU

Multiply unsigned

Multiplies $s by $t and stores the result in $LO.

v. JAL

Jump and link

Jumps to the calculated address and stores the return address in $31

vi. BNE

Branch on not equal

Branches if the two registers are not equal

vii. BLTZAL

Branch on less than zero and link

Branches if the register is less than zero and saves the return address in $31

viii. ADDIU

Add immediate unsigned (no overflow)

Adds two registers and stores the result in a register

ix. SW

Store word

The contents of $t is stored at the specified address.

x. MFHI

Move from HI

The contents of register HI are moved to the specified register.

9. Suppose we execute the following MIPS instructions.

```
   li $t0, 2
   li $t1, 5
   slt $t2, $t1, $t0
    beq $t2, $zero, skip
    addi $t1, $t2, 3
skip:
```

li $v0, 42

In the following table, write down value of each registers after program execution.

Register	$t0	$t1	$t2	$v0
Value	2	5	0	42

10. What are the various data types and registers available in MIPS programming?

Data types:

- Instructions are all 32 bits
- byte(8 bits), halfword (2 bytes), word (4 bytes)
- a character requires 1 byte of storage
- an integer requires 1 word (4 bytes) of storage

Registers:

- 32 general-purpose registers
- register preceded by $ in assembly language instruction
  two formats for addressing:
- using register number e.g. $0 through $31
- using equivalent names e.g. $t1, $sp
- special registers Lo and Hi used to store result of multiplication and division
- not directly addressable; contents accessed with special instruction mfhi ("move from Hi") and mflo ("move from Lo")
- stack grows from high memory to low memory

Register Number	Alternative Name	Description
0	zero	the value 0
1	$at	(**a**ssembler **t**emporary) reserved by the assembler
2-3	$v0 - $v1	(**v**alues) from expression evaluation and function results
4-7	$a0 - $a3	(**a**rguments) First four parameters for subroutine. Not preserved across procedure calls
8-15	$t0 - $t7	(**t**emporaries) Caller saved if needed. Subroutines can use w/out saving. Not preserved across procedure calls
16-23	$s0 - $s7	(**s**aved values) - Callee saved. A subroutine using one of these must save original and restore it before exiting. Preserved across procedure calls
24-25	$t8 - $t9	(**t**emporaries) Caller saved if needed. Subroutines can use w/out saving. These are in addition to $t0 - $t7 above. Not preserved across procedure calls.
26-27	$k0 - $k1	reserved for use by the interrupt/trap handler
28	$gp	**g**lobal **p**ointer. Points to the middle of the 64K block of memory in the static data segment.

29	$sp	**s**tack **p**ointer Points to last location on the stack.
30	$s8/$fp	**s**aved value / **f**rame **p**ointer Preserved across procedure calls
31	$ra	**r**eturn **a**ddress